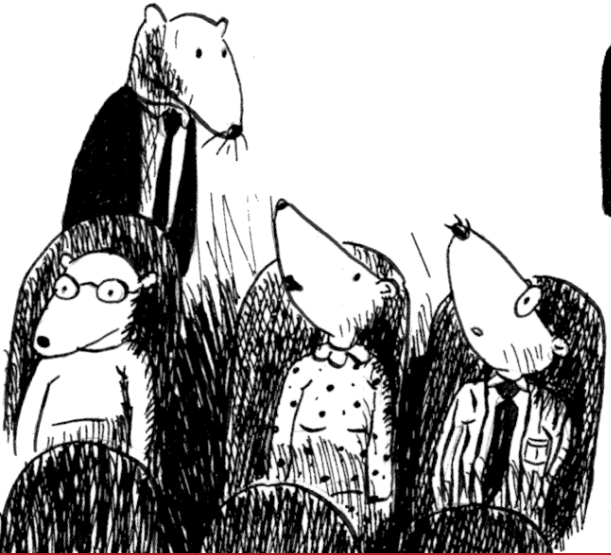


Aula 02
Machine Learning

Análise e Desenvolvimento de Sistemas

Eduardo Camargo de Siqueira

INSTITUTO FEDERAL
Brasília



drawn by adit.io

Este material foi criado por Eduardo Camargo de Siqueira e está disponível livremente para uso não comercial, para fins didáticos. Se você usar alguma parte deste material, adicione "copyright duca.pro".

Se você encontrar algum erro ou tiver alguma dúvida, envie um e-mail: eduardo.siqueira@ifb.edu.br. Obrigado por contribuir e espero que você aprenda bastante com este material!

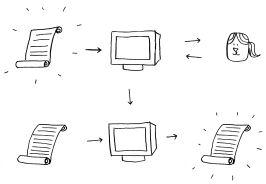
INSTITUTO FEDERAL
Brasília

Machine Learning

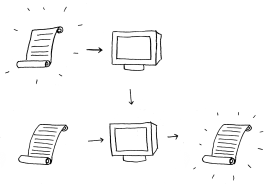
- Campo da Inteligência Artificial que permite que computadores aprendam padrões a partir de dados.
- Utiliza algoritmos para realizar previsões ou tomar decisões sem programação explícita.
- Aplicado em diversas áreas, como reconhecimento de imagem, análise preditiva e assistentes virtuais.

INSTITUTO FEDERAL
Brasília

Machine Learning



Aprendizado supervisionado - drawn by adit.io



Aprendizado não-supervisionado - drawn by adit.io



Aprendizado por reforço - drawn by adit.io

INSTITUTO FEDERAL
Brasília

Ferramentas

- Scikit-learn: Modelos clássicos de Machine Learning.
- TensorFlow e PyTorch: Deep Learning e Redes Neurais.
- Hugging Face: Modelos de NLP pré-treinados.



drawn by adit.io

INSTITUTO FEDERAL
Brasília

Machine Learning

- Qualidade dos dados influencia diretamente o desempenho.
- Overfitting: Modelo aprende padrões específicos demais do conjunto de treino.
- Explicabilidade dos modelos: Como garantir que as previsões sejam confiáveis?

INSTITUTO FEDERAL
Brasília

Modelos supervisionados

Regressão linear

- Algoritmo supervisionado usado para prever valores contínuos.
- Modelo matemático: $Y = aX + b$.
- Aplicações: Previsão de preços, crescimento populacional, análise de tendências.

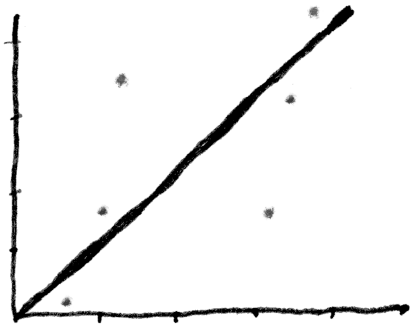
INSTITUTO FEDERAL
Brasília

Regressão linear

- Estabelece uma relação entre variável independente (X) e dependente (Y).
- Minimiza o erro entre a linha de regressão e os pontos de dados usando o método dos mínimos quadrados.
- Pode ser simples (uma variável independente) ou múltipla (mais de uma variável independente).

INSTITUTO FEDERAL
Brasília

Regressão linear



INSTITUTO FEDERAL
Brasília

Exemplo em Python

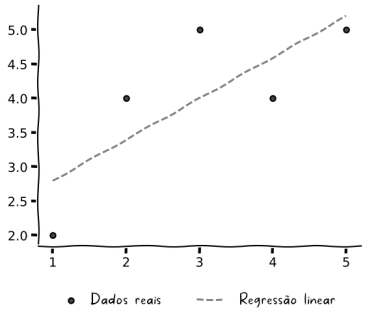
```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import
      LinearRegression

# Dados de exemplo
X = np.array([1, 2, 3, 4, 5]).reshape(-1, 1)
Y = np.array([2, 4, 5, 4, 5])

# Criando e treinando o modelo
modelo = LinearRegression()
modelo.fit(X, Y)

# Fazendo previsões
Y_pred = modelo.predict(X)

# Visualizando os resultados
plt.scatter(X, Y, color='blue', label='Dados
       Reais')
plt.plot(X, Y_pred, color='red', label='
      Regressão Linear')
plt.legend()
plt.show()
```



INSTITUTO FEDERAL
Brasília

Regressão logística

- Algoritmo de aprendizado supervisionado usado para resolver problemas de classificação binária.
- Diferente da regressão linear, que prevê valores contínuos, a regressão logística prevê probabilidades associadas a cada classe.

INSTITUTO FEDERAL
Brasília

Função sigmoide

- A regressão logística usa a **função sigmoide**:

$$\sigma(z) = \frac{1}{1 + \exp(-1)}$$

  Onde: $z = w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_n x_n$ é a combinação linear das variáveis de entrada.

- $\sigma(z)$ transforma o valor para um intervalo entre 0 e 1, interpretado como uma probabilidade.
- Se $\sigma(z) \geq 0.5$, classificamos como 1; caso contrário, classificamos como 0.

INSTITUTO FEDERAL
Brasília

Exemplo

Dado um conjunto de dados de pacientes com informações sobre pressão arterial e colesterol, queremos prever se um paciente tem risco alto de doença cardíaca (1) ou risco baixo (0).

INSTITUTO FEDERAL
Brasília

Implementação em Python

Importação das bibliotecas

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import
      train_test_split
from sklearn.preprocessing import
      StandardScaler
from sklearn.linear_model import
      LogisticRegression
from sklearn.metrics import accuracy_score,
      classification_report,
      confusion_matrix
```

Geração de dados sintéticos

```python
from sklearn.datasets import
      make_classification

# Criar dados fictícios para classificação
      binária
X, y = make_classification(n_samples=1000,
      n_features=2, n_classes=2,
                           n_informative=2,
       n_redundant=0, n_repeated=0,
      random_state=42)

# Convertendo para DataFrame
df = pd.DataFrame(X, columns=['
      Pressão_Arterial', 'Colesterol'])
df['Risco_Alto'] = y

# Visualizar amostra dos dados
print(df.head())
```

INSTITUTO FEDERAL
Brasília

Implementação em Python

Divisão de dados de treino e teste

```
X_train, X_test, y_train, y_test =
      train_test_split(X, y, test_size=0.2,
      random_state=42)

# Padronização dos dados para melhor
      desempenho do modelo
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Treinamento do modelo

```
# Criando e treinando o modelo de regressão
      logística
modelo = LogisticRegression()
modelo.fit(X_train, y_train)
```

INSTITUTO FEDERAL
Brasília

Implementação em Python

Avaliação do modelo

```python
# Predições
y_pred = modelo.predict(X_test)

# Acurácia
acc = accuracy_score(y_test, y_pred)
print(f'Acurácia do Modelo: {acc:.2f}')

# Relatório de classificação
print("Relatório de Classificação:")
print(classification_report(y_test, y_pred))

# Matriz de confusão
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel("Predito")
plt.ylabel("Real")
plt.show()
```

INSTITUTO FEDERAL
Brasília

Interpretação dos resultados

- Verdadeiros Positivos (TP): Casos corretamente classificados como risco alto.
- Falsos Positivos (FP): Casos classificados erroneamente como risco alto.
- Falsos Negativos (FN): Casos de risco alto classificados como risco baixo.
- Verdadeiros Negativos (TN): Casos corretamente classificados como risco baixo.



drawn by adit.io

INSTITUTO FEDERAL
Brasília

Métricas

- Precisão:

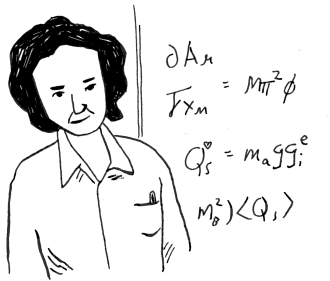
$$\text{precision} = \frac{TP}{TP + FP}$$

- Recall:

$$\text{recall} = \frac{TP}{TP + FN}$$

- F1-score (média harmônica):

$$\text{f1score} = 2 \times \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}}$$

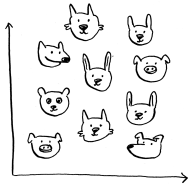


INSTITUTO FEDERAL
Brasília

Atividade

Adapte o código para usar um dataset real disponível no Kaggle para classificação binária.

Clique aqui



INSTITUTO FEDERAL
Brasília

Naïve Bayes

- Algoritmo de classificação baseado no Teorema de Bayes, assumindo que os atributos são independentes entre si.
- Amplamente utilizado para classificação de textos, detecção de spam, análise de sentimentos, entre outros.

INSTITUTO FEDERAL
Brasília

Implementação em Python

Importação da biblioteca

```
1 from sklearn.naive_bayes import GaussianNB
```

Treinamento do modelo

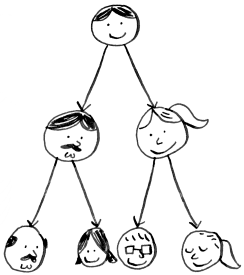
```
1 # Criando e treinando o modelo de Naïve Bayes Gaussiano
2 modelo = GaussianNB()
3 modelo.fit(X_train, y_train)
```

INSTITUTO FEDERAL
Brasília

Árvores de decisão

Árvores de decisão

- Modelos de aprendizado supervisionado usados para tarefas de classificação e regressão.
- São intuitivas, interpretáveis e poderosas, sendo amplamente utilizadas em problemas de tomada de decisão.
- A ideia principal é dividir os dados de forma recursiva, criando regras de decisão baseadas nas variáveis preditoras.



drawn by adit.io

INSTITUTO FEDERAL
Brasília

Árvores de decisão

Vantagens

- Fácil interpretação;
- Requer pouca preparação dos dados;
- Funciona para dados numéricos e categóricos.

Desvantagens

- Propensa a overfitting (se não limitada);
- Sensível a pequenas variações nos dados.

INSTITUTO FEDERAL
Brasília

Implementação em Python

Importação da biblioteca

```
from sklearn.tree import DecisionTreeClassifier, plot_tree
```

Treinamento do modelo

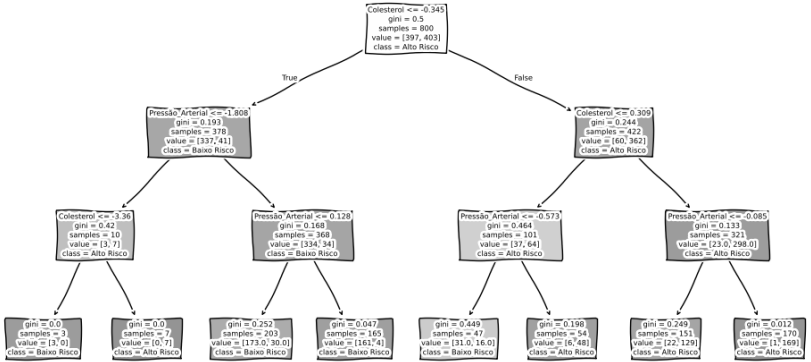
```
# Criando e treinando o modelo de árvore de decisão
modelo = DecisionTreeClassifier(criterion='gini', max_depth=3, random_state=42)
modelo.fit(X_train, y_train)
```

Visualização da árvore

```
plt.figure(figsize=(12,6))
plot_tree(modelo, feature_names=['Pressão_Arterial', 'Colesterol'], class_names=['Baixo
      Risco', 'Alto Risco'], filled=True)
plt.show()
```

INSTITUTO FEDERAL
Brasília

Visualização da árvore



INSTITUTO FEDERAL
Brasília

Random Forest

- Algoritmo de aprendizado supervisionado baseado em múltiplas árvores de decisão.
- Combina diversas árvores para obter um modelo mais preciso, robusto e resistente a overfitting.
- Cada árvore toma uma decisão independentemente.
  - No caso de classificação: a classe final é determinada por votação da maioria.
  - No caso de regressão: a previsão final é a média das previsões das árvores.



drawn by adit.io

INSTITUTO FEDERAL
Brasília

Implementação em Python

Importação da biblioteca

```
1 from sklearn.ensemble import RandomForestClassifier
```

Treinamento do modelo

```
1 # Criando e treinando o modelo Random Forest
2 modelo = RandomForestClassifier(n_estimators=100, max_depth=5, random_state=42)
3 modelo.fit(X_train, y_train)
```

INSTITUTO FEDERAL
Brasília

XGBoost (Extreme Gradient Boosting)

- Implementação avançada do algoritmo de Boosting.
- Altamente eficiente, rápido e tem sido amplamente utilizado em competições de Machine Learning.
- Boosting é uma técnica que combina vários modelos fracos em um modelo forte.
- Constrói sequencialmente modelos, corrigindo os erros dos anteriores.

INSTITUTO FEDERAL
Brasília

Boosting vs Bagging

- Bagging (ex: Random Forest): Treina vários modelos em paralelo e faz uma votação.
- Boosting (ex: AdaBoost, Gradient Boosting, XGBoost): Treina sequencialmente, ajustando erros da etapa anterior.

INSTITUTO FEDERAL
Brasília

Implementação em Python

Importação da biblioteca

```python
import xgboost as xgb
```

Treinamento do modelo

```python
model = xgb.XGBClassifier(
    n_estimators=100,  # Número de árvores
    learning_rate=0.1,  # Taxa de aprendizado
    max_depth=3,        # Profundidade máxima das árvores
    subsample=0.8,      # Amostragem de 80%
    colsample_bytree=0.8,  # Seleção de colunas
    objective='binary:logistic',  # Classificação binária
    eval_metric='logloss'  # Métrica de avaliação
)
model.fit(X_train, y_train)
```

INSTITUTO FEDERAL
Brasília

Support Vector Machine (SVM)

Support Vector Machine

- Algoritmo de aprendizado de máquina supervisionado usado para classificação e regressão.
- Amplamente utilizado devido à sua alta precisão e capacidade de generalização.
- O principal conceito é encontrar um hiperplano ótimo que separa os dados de diferentes classes no espaço de entrada.

INSTITUTO FEDERAL
Brasília

Support Vector Machine

- Algoritmo de aprendizado de máquina supervisionado usado para classificação e regressão.
- Amplamente utilizado devido à sua alta precisão e capacidade de generalização.
- O principal conceito é encontrar um hiperplano ótimo que separa os dados de diferentes classes no espaço de entrada.

INSTITUTO FEDERAL
Brasília

Tipos de SVM

- Linear: Usado quando os dados podem ser separados por uma reta.
- Não-linear: Utiliza funções kernel para transformar os dados em um espaço de maior dimensão, onde podem ser separados linearmente.

INSTITUTO FEDERAL
Brasília

Implementação em Python

Importação da biblioteca

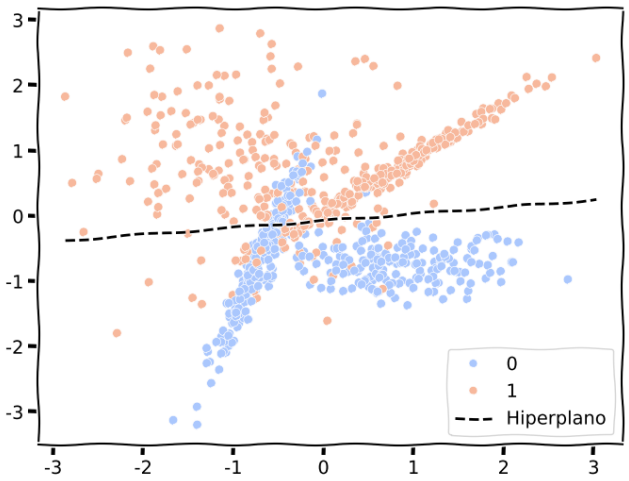
```
1 from sklearn.svm import SVC
```

Treinamento do modelo

```
1 # Criando e treinando um modelo SVM com kernel linear
2 modelo = SVC(kernel='linear', C=1.0)
3 modelo.fit(X_train, y_train)
```

INSTITUTO FEDERAL
Brasília

Visualização do hiperplano



INSTITUTO FEDERAL
Brasília

Atividade

Adapte os códigos para usar o dataset 'Predição de Insuficiência Cardíaca', disponibilizado no Kaggle. Qual foi o melhor resultado? Tente encontrar razões para o desempenho de cada modelo.

INSTITUTO FEDERAL
Brasília

"Compreender o mundo envolve encontrar padrões nos dados"

- Geoffrey Hinton



drawn by adit.io

INSTITUTO FEDERAL
Brasília

FIM

- Dúvidas!?
- http://duca.pro.br/ | eduardo.siqueira@ifb.edu.br
- Ilustrações: copyright Manning Publications, drawn by adit.io
- Obrigado pela atenção!

INSTITUTO FEDERAL
Brasília